

GamutX

Operators

Python Basics

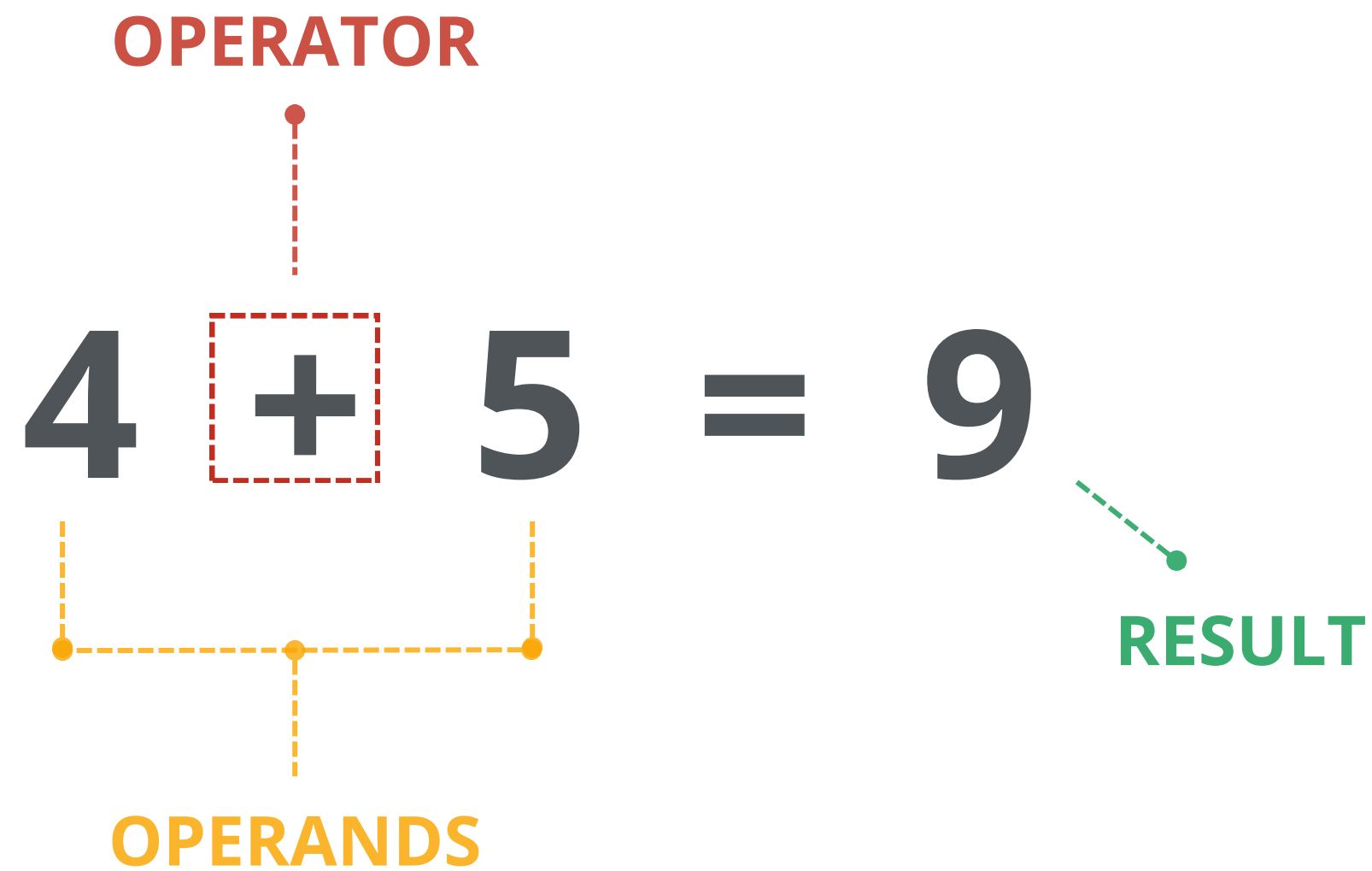


Introduction to Operators

- What is an Operator?
- Types of Operators

What is an Operator?

An **operator** is basically a **symbol or a keyword** that lets us **perform some action** on **values & variables**



An **operand** is just a **fancy name** for the **values or variables**

Types of Operators

- We can group operators into several categories depending on what they do.

Arithmetic Operator

Assignment Operator

Comparison Operator

Logical Operator

Identity Operator

Bitwise Operator

Membership Operator

Note: Identity, Bitwise , Membership Operators are covered in Intermediate Level

Arithmetic Operators

- What is an Arithmetic Operator?
- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Floor Division (//)
- Modulus (%)
- Exponentiation (**)

What is an Arithmetic Operator?

- Arithmetic operators are used to **perform basic math operations**.
- These operators work with numbers, **such as integers (int) and decimals (float)**.
- You'll commonly **use them to add, subtract, multiply, divide, and more**.

Arithmetic Operators List

Addition (+)

Subtraction (-)

Multiplication (*)

Division (/)

Exponentiation (**)

Modulus (%)

Floor Division (//)

Addition (+)

- The **+** operator is used to add two numbers together.

Basic Check

```
a = 10
b = 3
result = a + b
print(result) # Output: 13
```

Diagram illustrating the addition of two integers (10 and 3) using the addition operator (+). The result is 13.

Adding Negative Numbers

```
# Defining two negative numbers
a = -5
b = -10

# Using the addition operator
result = a + b -5 + (-10)

# Printing the result
print(result) # -15
```

Adding Integers & Floats

```
# Defining an integer and a float
a = 5 -----> Integer Data
b = 2.5 -----> Float Data

# Using the addition operator
result = a + b

# Printing the result
print(result) # 7.5
```

Adding Floating Numbers

```
# Defining two float variables
x = 2.5
y = 3.5 -----> Float Data Type

# Using the addition operator
result = x + y

# Printing the result
print(result) # 6.0
```

Adding Multiple Values

```
# Adding multiple integers
a = 10
b = 20
c = 30

# Using the addition operator
total = a + b + c
total =  $\frac{a}{10} + \frac{b}{20} + \frac{c}{30}$ 

# Printing the result
print(total) #60
```

Subtraction (-)

- The **subtract operator (-)** is used to find the **difference** between two numbers.

Basic Check

```
a = 10
b = 3
result = a - b
print(result) # Output: 7
```

Negative Numbers

```
# Defining two negative numbers
a = -10
b = -5

# Using the subtraction operator
result = a - b

# Printing the result
print(result) # -5
```

Subtracting Multiple Values

```
# Defining three variables
a = 100
b = 30
c = 10

# Using the subtraction operator
result = a - b - c

# Printing the result
print(result) # 60
```

Subtracting an Integer from a Float

```
# Defining a float and an integer
x = 10.5 ---> Float Data Type
y = 3 ---> Int Data Type

# Using the subtraction operator
result = x - y

# Printing the result
print(result) # 7.5
```

Floating-Point Numbers

```
# Defining two float variables
x = 5.5
y = 2.5 ---> Float Data Type

# Using the subtraction operator
result = x - y

# Printing the result
print(result) # 3.0
```

Multiplication (*)

- The ***** operator multiplies two numbers.

Multiplies two numbers.

```
a = 10
b = 3
result = a * b
print(result) # Output: 30
```

Floating-Point Numbers

```
# Defining two float variables
x = 2.5
y = 3.0

# Using the multiplication operator
result = x * y
print(result) # Output: 7.5
```

Multiplying Multiple Values

```
# Defining three variables
a = 2
b = 3
c = 4

# Using the multiplication operator
result = a * b * c
print(result) # Output: 24
```

Integer and a Float

```
# Defining an integer and a float
x = 10
y = 0.5

# Using the multiplication operator
result = x * y
print(result) # Output: 5.0
```

Negative Numbers

```
# Defining two negative numbers
a = -3
b = -7

# Using the multiplication operator
result = a * b
print(result) # Output: 21
```

Division (/)

- **Divides** the first number by the second and **returns a decimal value (float)**.

Returns the Quotient as an output

```
a = 10
b = 3
result = a / b
print(result) # Output: 3.333...
```

Floating-Point Numbers

```
# Defining two float variables
x = 7.5
y = 2.5

# Using the division operator
result = x / y
print(result) # Output: 3.0
```

Integer and a Float

```
# Defining a float and an integer
x = 10.0 # Float Data Type
y = 4    # Int Data Type

# Using the division operator
result = x / y
print(result) # Output: 2.5
```

Multiplying Multiple Values

```
# Defining three variables
a = 100
b = 5
c = 2

# Using the division operator
result = a / b / c
print(result) # Output: 10.0
```

Negative Numbers

```
# Defining a positive and a ( - ) number
a = 15
b = -3

# Using the division operator
result = a / b
print(result) # Output: -5.0
```

Floor Division (//)

GamutX

- **Divides** the first number by the second and **returns the whole number part only (no decimal).**

Returns the Quotient and round it to int

```
a = 10
b = 3
result = a // b
print(result) # Output: 3
```

Floating-Point Numbers

```
# Defining two float variables
x = 7.5
y = 2.5

# Using the floor division operator
result = x // y
print(result) # Output: 3.0
```

Multiplying Multiple Values

```
# Defining three variables
a = 100
b = 5
c = 2

# Using the floor division operator
result = a // b // c
print(result) # Output: 10
```

Integer and a Float

```
# Defining a float and an integer
x = 10.5 # Float Data Type
y = 4

# Using the floor division operator
result = x // y
print(result) # Output: 2.0
```

Negative Numbers

```
# Defining a positive and a negative number
a = 15
b = -4

# Using the floor division operator
result = a // b
print(result) # Output: -4
```

Modulus (%)

GamutX

- **Returns the remainder** after dividing the first number by the second.

Returns the remainder as an output

```
a = 10
b = 3
result = a % b
print(result) # Output: 1
```

Negative Number

```
a = 15
b = -4
result = a % b
print(result) # Output: 3
```

Large Numbers

```
a = 100000
b = 333
result = a % b
print(result) # Output: 100
```

Floating-Point Numbers

```
x = 7.5
y = 2.5
result = x % y
print(result) # Output: 0.0
```

Modulus with Variables of Different Types

```
a = 9
b = 2.5
result = a % b
print(result) # Output: 2.0
```

Exponentiation (**)

- Raises the first number to the power of the second.

Basic Example

```
a = 10  
b = 3  
result = a ** b  
print(result) # Output: 1000
```

10³

result = (10 * 10 * 10)
= 1000

Floating-Point Numbers

```
# Defining two float variables  
x = 2.5  
y = 2.0  
  
# Using the exponential operator  
result = x ** y  
print(result) # Output: 6.25
```

Zero Exponent

```
# Defining a variable  
a = 5  
  
# exponential with zero exponent  
result = a ** 0  
print(result) # Output: 1
```

Concept Practice Task

GamutX

Fractional Exponents

```
# Defining a number and a fractional exponent
x = 16
y = 0.5

# Using the exponential operator
result = x ** y
print(result) # Output: 4.0
```

Negative Base

```
# Defining a - base and a + exponent
a = -3
b = 2

# Using the exponential operator
result = a ** b
print(result) # Output: 9
```

Exponential with Negative Exponent

```
a = 2
result = a ** -2
print(result)

# Output: 0.25
```

Exponential with Large Numbers

```
a = 10
b = 10

result = a ** b
print(result)

# Output: 10000000000
```

Comparison Operators

- What is a Comparison Operator?
- Equal To (==)
- Not Equal To (!=)
- Greater Than (>)
- Less Than (<)
- Greater Than or Equal To (>=)
- Less Than or Equal To (<=)

What is a Comparison Operator?

- Comparison operators are **used to compare two values**.
- They return either:
 - **True** → if the comparison **is correct**
 - **False** → if the comparison **is not correct**
- These operators are often **used in conditions like if statements**.

List of Comparison Operator

== Equal to

!= Not equal to

> Greater than

< Less than

>= Greater than or equal to

<= Less than or equal to

Equal To (==)

- Checks if two values are equal.
- Returns True if they are the same, otherwise False.

Comparing Integers

```
a = 10
b = 10
result = a == b
print(result) # Output: True
```

Floating-Point Numbers

```
# Defining two float variables
x = 2.5
y = 2.5

result = (x == y)
print(result) # Output: True
```

Boolean Values

```
# Defining two boolean variables
flag1 = True
flag2 = False

result = (flag1 == flag2)
print(result) # Output: False
```

Different Data Types

```
# Defining a string and an integer
a = 10
b = "10" String Data Type

result = (a == b)
print(result) # Output: False
```

Not Equal To (!=)

- Checks if two values are not equal.
- Returns True if they are different, otherwise False.

Comparing Integers

```
a = 10
b = 5
result = a != b
print(result) # Output: True
```

Floating-Point Numbers

```
# Defining two float variables
x = 2.5
y = 2.5

result = (x != y)
print(result) # Output: False
```

Boolean Values

```
# Defining two boolean variables
flag1 = True
flag2 = False

result = (flag1 != flag2)
print(result) # Output: True
```

Different Data Types

```
# Defining an integer and a string
a = 10
b = "10" -----> String Data

result = (a != b)
print(result) # Output: True
```

Greater Than (>)

- Checks if the first value is greater than the second.

Comparing Integers

```
a = 10
b = 5
result = a > b
print(result) # Output: True
```

Floating-Point Numbers

```
# Defining two float variables
x = 7.5
y = 8.3

result = (x > y)
print(result) # Output: False
```

Boolean Values

```
# Defining two boolean variables
flag1 = True
flag2 = False

result = (flag1 > flag2)
print(result) # Output: True
```

Negative Number

```
# Defining two negative numbers
a = -3
b = -5

result = (a > b)
print(result) # Output: True
```

Less Than (<)

- Checks if the first value is less than the second.
- Returns True if it's smaller, otherwise False.

Comparing Integers

```
a = 5
b = 10
result = a < b
print(result) # Output: True
```

Floating-Point Numbers

```
# Defining two float variables
x = 3.2
y = 2.5

result = (x < y)
print(result) # Output: False
```

Negative Number

```
# Defining two negative numbers
a = -5
b = -3

result = (a < b)
print(result) # Output: True
```

Boolean Values

```
# Defining two boolean variables
flag1 = False
flag2 = True

result = (flag1 < flag2)
print(result) # Output: True
```

Greater Than or Equal To ($\geq$)

- Checks if the first value is greater than or equal to the second.
- Returns True if it's either greater or equal.

Comparing Integers

```
a = 10
b = 10
result = a >= b
print(result) # Output: True
```

Floating-Point Numbers

```
# Defining two float variables
x = 7.5
y = 7.5

result = (x >= y)
print(result) # Output: True
```

Negative Number

```
# Defining two negative numbers
a = -3
b = -5

result = (a >= b)
print(result) # Output: True
```

Concept Practice Task

Different Data Types

```
# Defining an integer and a string
a = 10
b = "10" -----> String Data Type

result = (a >= b)
print(result)
# Output: TypeError (cannot compare int and str)
```

Greater Than or Equal to equal to with Zero

```
a = 0

result = (a >= 0)
print(result) # Output: True
```

Less Than or Equal To (<=)

- Checks if the first value is **less than or equal to** the second.
- Returns True if it's **either less or equal**.

Comparing Integers

```
a = 5
b = 10
result = a <= b
print(result) # Output: True
```

Floating-Point Numbers

```
# Defining two float variables
x = 3.5
y = 3.5

result = (x <= y)
print(result) # Output: True
```

Negative Number

```
# Defining two negative numbers
a = -5
b = -3

result = (a <= b)
print(result) # Output: True
```

Concept Practice Task

Different Data Types

```
# Defining an integer and a string
a = 5
b = "5" -----> String Data Type

result = (a <= b)
print(result)
# Output: TypeError (cannot compare int and str)
```

Less Than or Equal to with Zero

```
a = 0

result = (a <= 0)
print(result) # Output: True
```

Summary

Operator	What it Does	Example	Output
+	Adds numbers	10 + 3	13
-	Subtracts numbers	10 - 3	7
*	Multiplies numbers	10 * 3	30
/	returns float (division)	10 / 3	3.33
//	return int (floor Division)	10 // 3	3
%	Return Remainder (modulus)	10 % 3	1
**	Power (exponentiation)	10 ** 3	1000

Logical Operators

- What is a Logical Operator?
- AND Operator (and)
- OR Operator (or)
- NOT Operator (not)

What is a Logical Operator?

- Logical operators are **used to combine multiple conditions**.
- They **return** either:
 - **True** if the **combined condition is satisfied** , **False** if it is not
- **Precedence** of logical operator are **Logical not, Logical and, Logical or**.

List of Logical Operators are

- **and**: Both conditions must be true.
- **or**: At least one condition must be true.
- **not**: Negates the condition.

and Operator

- **returns True** if both conditions are **true**. If either **condition** is false, it returns **False**

AND with Booleans

```
a = True
b = False
result = a and b
print(result) # Output: False
```

AND with Numeric Conditions

```
# Defining variables
x = 10
y = 20

result = (x < 15) and (y > 15)
print(result) # Output: True
```

AND with Comparison Operators

```
# Defining a variable
score = 85

is_pass = (score >= 50) and (score <= 100)
print(is_pass) # Output: True
```

Concept Practice Task

AND with Conditions

```
age = 25
has_license = True

can_drive = age >= 18 and has_license
print(can_drive) # Output: True
```

AND with Multiple Conditions

```
temperature = 70
is_raining = False

go_outside = temperature > 65 and not is_raining
print(go_outside) # Output: True
```

OR Operator (or)

- **returns True** if either expression is **True**; it **returns False only** if both are **False**.

OR with Booleans

```
a = True
b = False
result = a or b
print(result) # Output: True
```

OR with Numeric Conditions

```
# Defining variables
x = 5
y = 20

result = (x < 10) or (y < 15)
print(result) # Output: True
```

OR with Comparison Operators

```
# Defining a variable
score = 40

is_failing = (score < 50) or (score == 0)
print(is_failing) # Output: True
```

OR with Multiple Conditions

```
# Defining variables
temperature = 50
is_sunny = True

go_outside = temperature > 65 or is_sunny
print(go_outside) # Output: True
```

NOT Operator (not)

- **negates a boolean expression**, returning **False** if it's True and True if it's False.

NOT with Booleans

```
a = True
result = not a
print(result) # Output: False
```

NOT with Numeric Conditions

```
# Defining a variable
x = 5

result = not (x == 10)
print(result) # Output: True
```

Chaining NOT Operators

```
# Defining a variable
condition = False

result = not not condition
print(result)
# Output: False
```

NOT with Multiple Conditions

```
# Defining variables
age = 16
has_permission = False

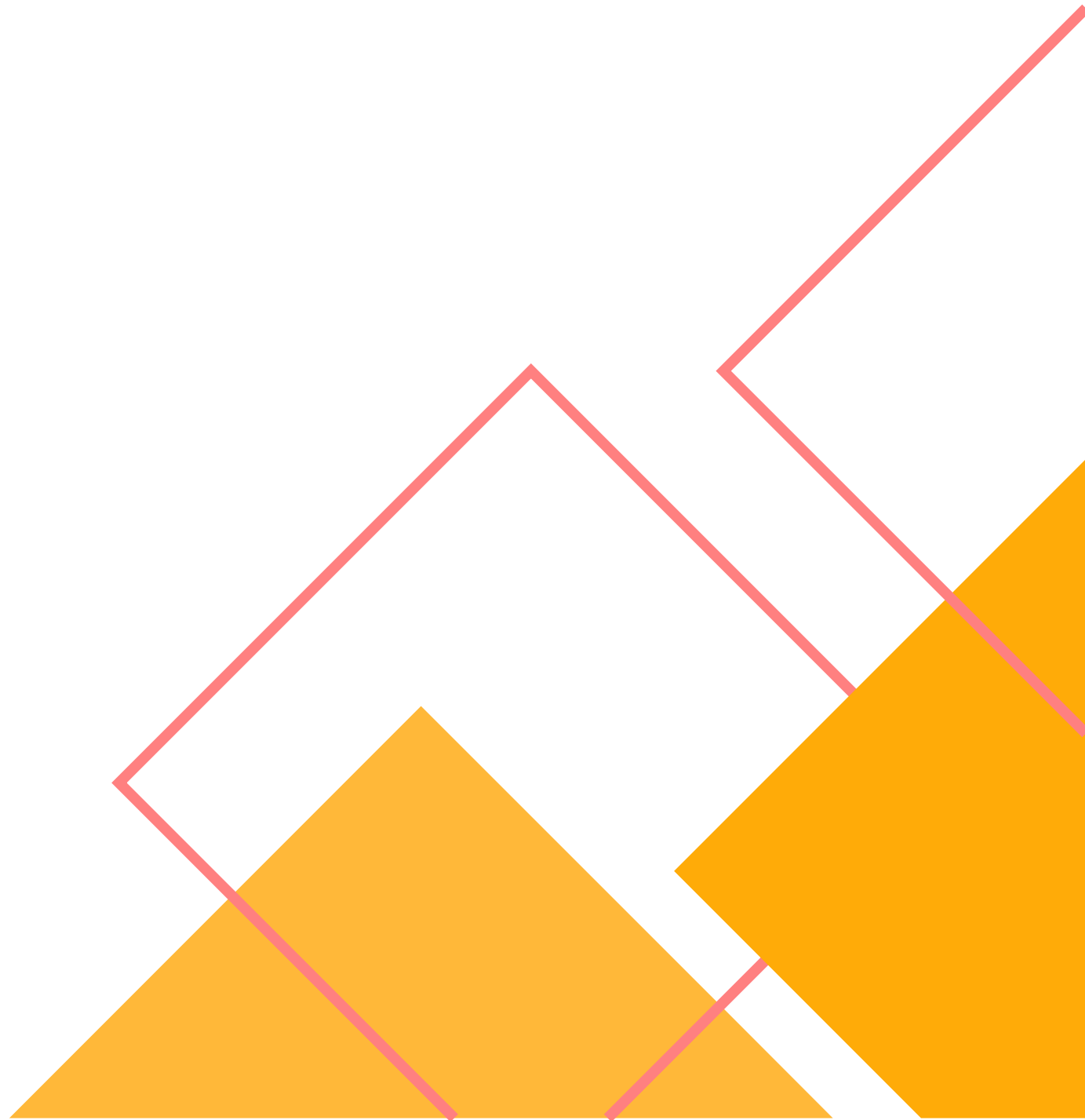
can_drive = not (age >= 18 or has_permission)
print(can_drive) # Output: True
```

Concept Practice Task

NOT with Comparison Operators

```
# Defining a variable  
score = 60  
  
is_failing = not (score >= 50)  
print(is_failing) # Output: False
```

Operator Precedence

- What is Operator Precedence?
 - Understanding the PEMDAS Rule
 - P — Parentheses
 - E — Exponentiation
 - Multiplication and Division
 - A/S — Addition and Subtraction
 - Combined Examples
 - Complex Expression
- 

What is Operator Precedence?

- 📌 Operator precedence determines the **order in which operators are evaluated in an expression**
- 📌 **Higher precedence** operators are evaluated **before lower precedence ones**.

BODMAS Rule

Brackets

→

(or)

←

Order

→

✓ or x^2

←

Division

→

÷ or ×

←

Multiplication

→

× or ÷

←

Addition

→

+

←

Subtraction

→

-

←

PEMDAS Rule

----->

Python Follows this

Parenthesis

Exponents

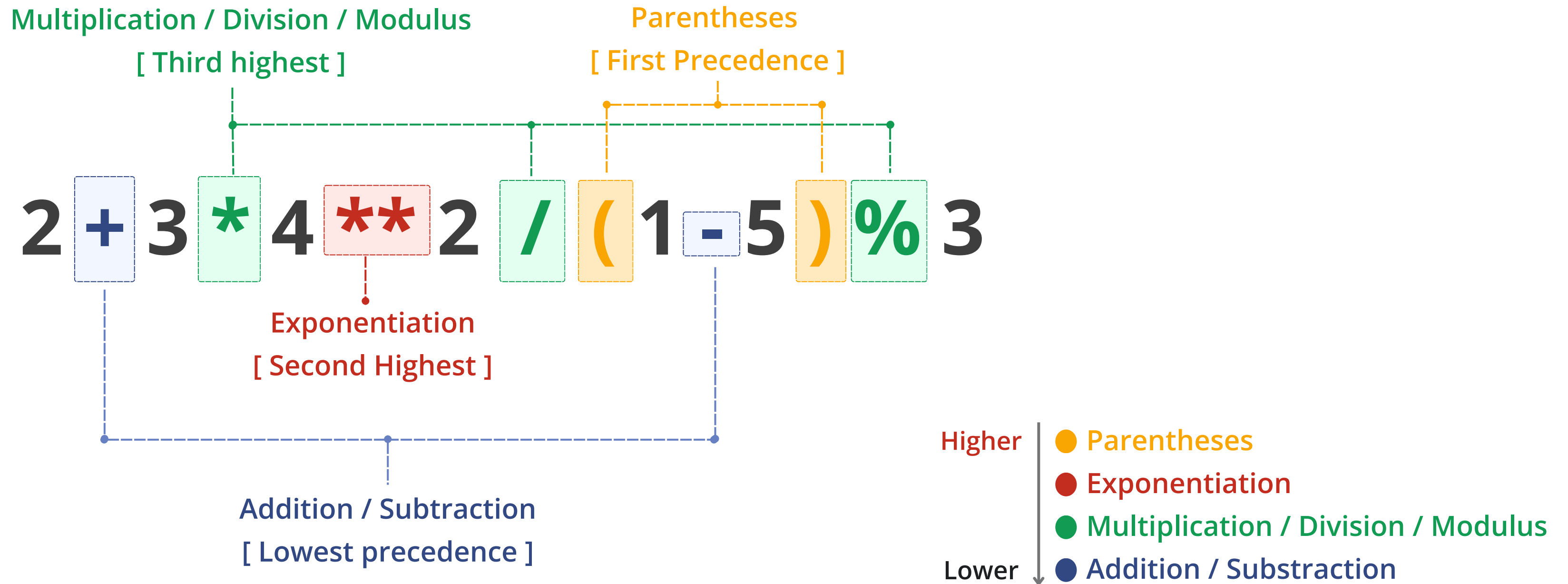
Multiplication

Division

Addition

Subtraction

Understanding the PEMDAS Rule



Note: When operators have equal precedence, Python evaluates left to right

P – Parentheses

- Parentheses have the **highest precedence and are evaluated first**
- regardless of **where they are in the expression**.

```
result = (2 + 3) * 4
```

```
print(result) # Output: 20
```

How it Work

- **first:** $2 + 3 = 5$,
- **then** $5 * 4 = 20$

E – Exponentiation

Exponentiation is evaluated next after parentheses.

```
result = 2 ** 3 + 1
```

```
print(result) # Output: 9
```

How it Work

- **first:** $2 ** 3 = 8$
- **then** $8 + 1 = 9$

Multiplication *, Division /, Floor Division //, Modulus %

- Multiplication, Division, Floor Division, and Modulus **have the same precedence**,
- Python **evaluates them from left to right** when they **appear in the same expression**

◆ Multiplication * Example

```
result = 5 * 4 + 6  
print(result) # Output: 26
```

First $5 * 4 = 20$, then $20 + 6 = 26$

◆ Floor Division // Example

```
result = 20 // 4 * 2  
print(result) # Output: 10
```

First: $20 // 4 = 5$, then $5 * 2 = 10$

◆ Division / Example

```
result = 10 / 2 * 5  
print(result) # Output: 25.0
```

First: $10 / 2 = 5.0$, then $5.0 * 5 = 25.0$

◆ Modulus % Example

```
result = 10 % 3 + 4  
print(result) # Output: 5
```

First: $10 \% 3 = 1$ (that's the remainder), then $1 + 4 = 5$

A/S – Addition and Subtraction

- Addition and subtraction also **have the same precedence**, and like **multiplication and division**, they are **evaluated left to right**.

◆ Subtraction First, Then Addition

```
result = 10 - 2 + 3  
print(result) # Output: 11
```

First : $10 - 2 = 8$, then $8 + 3 = 11$

◆ Addition First, Then Subtraction

```
result = 5 + 2 - 3  
print(result) # Output: 4
```

First : $5 + 2 = 7$, then $7 - 3 = 4$

Concept Practice Task

GamutX

Simple Parentheses

```
result = 6 * (2 + 3)
print(result)
# Output:
# 30 (2 + 3 = 5, 6 * 5 = 30)
```

Complex Exponents

```
result = 5 + 2 ** 3 * 3
print(result)
# Output:
# 29 (2^3 = 8, 8 * 3 = 24, 5 + 24 = 29)
```

Division and Addition

```
result = 10 / 2 + 5
print(result)
# Output:
# 10.0 (10/2 = 5, 5 + 5 = 10)
```

Nested Operations

```
result = (5 + (2 * 3)) ** 2 - 4
print(result)
# Output:
# 61 (2*3=6, 5+6=11, 11^2=121, 121-4=117)
```

Concept Practice Task

GamutX

Multiplication Before Addition

```
result = 7 + 2 * 3
print(result)
```

Output:
13 (2*3=6, 7+6=13)

Order of Operations with Division

```
result = 10 - 5 / 5 + 3
print(result)
```

Output:
12.0 (5/5=1, 10-1=9, 9+3=12)

Exponents with Parentheses

```
result = (3 + 2) ** 2 * 2
print(result)
```

Output:
50 (3+2=5, 5^2=25, 25*2=50)

All Operations in One Expression

```
result = (2 + 3) * (5 - 2) ** 2 + 4 / 2
print(result)
```

Output:
43.0 ((2+3)=5, (5-2)=3, 3^2=9, 5*9=45, 4/2=2, 45+2=47)

Concept Practice Task

Use of Zero

```
result = 5 * (10 - 10) + 1  
print(result)
```

```
# Output:  
# 1 (10-10=0, 5*0=0, 0+1=1)
```

Order of Operations with Mixed Types

```
result = 7.5 + (3 * 2) - (4 / 2)  
print(result)
```

```
# Output:  
# 12.0 (3*2=6, 4/2=2, 7.5+6-2=11.5)
```

Negative Numbers

```
result = -3 + (4 * -2) + 5  
print(result)
```

```
# Output:  
# -5 (-2*4=-8, -3-8=5)
```

Chaining Operations with Variables

```
x = 5  
y = 3  
result = (x + y) ** 2 - (x * y) + (x / y)  
print(result)
```

```
# Output:  
# 50.666666666666664
```

Combined Examples

◆ Multiplication and Division

```
result = 15 + 3 * 2 - 6 / 3  
print(result) # Output: 19.0
```

◆ Division and Modulus

```
result = 10 / 2 + 5 % 3  
print(result) # Output: 7.0
```

◆ Combination of Multiplication, Division, and Modulus

```
result = 8 * 3 / 4 + 10 % 3  
print(result) # Output: 7.0
```

Complex Expression

Mix of All Operators

```
result = (10 + 5 * 2) % 3 + 4 ** 2 / 2 - (8 - 5)
print(result) # Output: 7
```

Solve First Bracket:

Multiply First: $5 * 2 = 10$ Then Add: $10 + 10 = 20$

$result = (20) \% 3 + 4 ** 2 / 2 - (8 - 5)$

Solve Second Bracket:

$8 - 5 = 3$

$result = (20) \% 3 + 4 ** 2 / 2 - 3$

Evaluate $4 ** 2$:

$4 ** 2 = 16$

$result = (20) \% 3 + 16 / 2 - 3$

Calculate $20 \% 3$:

$20 \% 3 = 2$

$result = 2 + 16 / 2 - 3$

Divide $16 / 2$:

$16 / 2 = 8$

$result = 2 + 8 - 3$

Final Step :

Add:

$2 + 8 = 10$

Subtract:

$10 - 3 = 7$

$result = 7$

◆ Nested Subtraction with Division

```
result = 3 + 4 * 2 - (1 - 5) / 2  
print(result) # Output: 13.0
```

◆ Parentheses and Exponentiation

```
result = (2 + 3) * (4 - 1) ** 2 / 5  
print(result) # Output: 9.0
```

◆ Multiplication and Parentheses

```
result = 10 - 2 * 5 + (8 / 4) ** 2  
print(result) # Output: 4
```

◆ Negative Values and Exponents

```
result = (3 + 5) * (2 - 4) + 7 ** 2 / 7  
print(result) # Output: -9.0
```

Common Errors with Operators

- TypeError
- Syntax Errors
- Division by Zero
- Improper Use of Parentheses

TypeError

- Occurs when **operators** are used on incompatible data types.

✗ Incorrect

```
a = "5" ←--- String
b = 10 ←--- int
result = a + b
```



```
PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/Users/HP/Desktop/tt/test.py
Traceback (most recent call last):
  File "c:\Users\HP\Desktop\tt\test.py", line 3, in <module>
    result = a + b
TypeError: can only concatenate str (not "int") to str
PS C:\Users\HP\Desktop\tt>
```

TypeError

✓ Correct

```
a = 5 ←--- int
b = 10
result = a + b
print(result) # Output: 15
```

Syntax Errors

- Occurs when the **syntax** of the **expression** is incorrect.

✗ Incorrect

```
a = 10  
b = 5  
result = a +
```

Incomplete Expression

✓ Correct

```
a = 10  
b = 5  
result = a + b  
print(result) # Output: 15
```

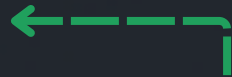
```
PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/Users  
⊗ File "c:\Users\HP\Desktop\tt\test.py", line 3  
    result = a +  
                ^  
SyntaxError: invalid syntax  
○ PS C:\Users\HP\Desktop\tt>
```

Syntax Error

Division by Zero

- Occurs when trying to **divide by zero**, which **is not allowed in Python**.

```
a = 10  
b = 0  
result = a / b
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.exe" c:/User  
Traceback (most recent call last):  
  File "c:\Users\HP\Desktop\tt\ttest.py", line 3, in <module>  
    result = a / b  
ZeroDivisionError: division by zero  
PS C:\Users\HP\Desktop\tt>
```

ZeroDivisionError

Improper Use of Parentheses

- Forgetting to use parentheses** can lead to **unexpected results** due to operator precedence.

✗ Incorrect

```
result = 10 + 2 * 5  
print(result) # Output: 20
```

✓ Correct

```
result = (10 + 2) * 5  
print(result)  
# Output: 60
```

Thank You

